

Introduction to Clustering

Clustering is a type of unsupervised learning algorithm. Its goal is to look at a number of data points and automatically find points that are related or similar to each other, grouping them into "clusters."

Supervised vs. Unsupervised Learning

To understand clustering, it helps to contrast it with the supervised learning methods (like logistic regression) you learned previously.

Feature	Supervised Learning	Unsupervised Learning
Data	Includes input features (x) and target labels (y) .	Includes only input features (x) . No target labels are provided.
Visualization	Data points are distinct (e.g., x 's and y 's).	Data points look the same (e.g., all dots).
Goal	Find a decision boundary to predict the "right answer" (y).	Find "interesting structure" or patterns within the data.
Guidance	We tell the algorithm what the correct output is.	We ask the algorithm to figure it out on its own.

- **In the plot above:** You can see how an unsupervised algorithm looks at raw data (without labels) and identifies distinct groups (clusters) based on the proximity of the points to one another.

Real-World Applications of Clustering

Clustering is widely used across many industries to find hidden structures in data:

- **News Aggregation:** Grouping similar news articles together (e.g., grouping all stories about "Pandas" or a specific event) to present them as a single topic.
- **Market Segmentation:** Analyzing a database of customers to discover distinct groups.
 - *Example:* Deeplearning.ai discovered groups such as "Career Developers," "Skill Growers," and "AI Updaters" to better tailor content.
- **DNA Analysis:** Analyzing genetic expression data to group individuals who exhibit similar genetic traits.
- **Astronomy:** Analyzing astronomical data to group celestial bodies together to identify galaxies or coherent structures in space.

How K-Means Clustering Works

K-means is an iterative algorithm that automatically groups unlabeled data into clusters. It works by repeatedly performing two specific steps until it finds the best grouping.

The Algorithm

1. Initialization:

- The algorithm starts by taking a **random guess** at where the centers of the clusters might be.
- It picks K random points (where K is the number of clusters you want to find). These points are called **Cluster Centroids**.

2. The Iterative Loop:

K-means repeatedly cycles through two main steps:

- **Step 1: Assign points to cluster centroids**
 - The algorithm goes through every data point ($x^{(1)}$ to $x^{(m)}$).
 - For each point, it calculates the distance to the centroids.
 - It assigns (or "colors") the point to the cluster centroid it is **closest to**.
- **Step 2: Move cluster centroids**
 - The algorithm looks at all the points assigned to a specific cluster (e.g., all the red points).
 - It calculates the **average (mean)** location of those points.
 - It moves the centroid to that new average location.

Convergence

- After moving the centroids (Step 2), the algorithm goes back to Step 1 and re-assigns points based on the *new* centroid locations. Some points might change clusters (change color) because the centroids have moved.
- This cycle (Assign → Move → Assign → Move) repeats.
- **Convergence** occurs when there are no more changes:
 - Points stop switching clusters (colors remain the same).
 - Centroids stop moving (their location is stable).

At this point, the algorithm has successfully grouped the data into distinct clusters.

The K-Means Algorithm

The algorithm takes a dataset $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$ and the number of clusters K as input.

Step 1: Random Initialization

Randomly initialize K cluster centroids: $\mu_1, \mu_2, \dots, \mu_K$.

- These centroids are vectors with the same dimension (n) as your training examples.
- *Example:* If your data has 2 features (like height and weight), each centroid μ will also be a vector of 2 numbers.

Step 2: The Iterative Loop

The algorithm repeatedly performs these two steps until convergence:

A. Assign Points to Cluster Centroids

For every training example $x^{(i)}$ (from $i = 1$ to m), determine which centroid is closest.

- We calculate the distance using the standard Euclidean distance (L2 norm): $\|x^{(i)} - \mu_k\|$.
- *Note:* In code, it is computationally convenient to minimize the **squared** distance: $\|x^{(i)} - \mu_k\|^2$.
- We set a variable $c^{(i)}$ to be the index (1 to K) of the closest cluster centroid.

$c^{(i)} := \text{index } k \text{ that minimizes } \|x^{(i)} - \mu_k\|^2$

B. Move Cluster Centroids

For every cluster k (from 1 to K), update the centroid to be the **average (mean)** of all points assigned to that cluster.

- *Example:* If cluster 1 has assigned points $x^{(1)}, x^{(5)}, x^{(6)}, x^{(10)}$, the new centroid location is:

$$\mu_1 = \frac{1}{4}(x^{(1)} + x^{(5)} + x^{(6)} + x^{(10)})$$

Corner Case: Empty Clusters

What happens if a cluster has **zero** points assigned to it during the assignment step? (e.g., trying to average 0 points).

1. **Eliminate the cluster (Most Common):** simply remove that centroid. You will end up with $K - 1$ clusters.
 2. **Reinitialize:** If you strictly require K clusters, you can randomly re-initialize that specific centroid and run the loop again.
-

Application: Non-Separated Data

K-means is often used on data that looks like a single "blob" rather than distinct, separated islands.

- **Example: T-Shirt Sizing**

- **Data:** Heights and Weights of customers (continuous distribution).
- **Goal:** Determine optimal sizing for Small, Medium, and Large.
- **Process:** Run K-means with $K = 3$.
- **Result:** The algorithm groups the data into three clusters. The centroids (μ_1, μ_2, μ_3) represent the "prototypical" measurements for the three sizes, ensuring the shirts fit the largest number of people effectively.

The K-Means Cost Function

K-means is not just moving points around randomly; it is mathematically optimizing a specific cost function.

The Objective (Minimizing Distortion)

The cost function J (also called the **Distortion Function**) measures the sum of squared distances between every training example and its assigned cluster centroid.

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

- $x^{(i)}$: The i -th training example.
- $c^{(i)}$: The index of the cluster to which $x^{(i)}$ is currently assigned.
- $\mu_{c^{(i)}}$: The location of the centroid for that cluster.
- $\|\dots\|^2$: The squared Euclidean distance.

Goal: The algorithm tries to find the assignments (c) and the centroid locations (μ) that minimize this cost function J .

How the Algorithm Minimizes J

The two steps of the K-means algorithm correspond exactly to minimizing this function with respect to different variables:

1. Step 1 (Assign Points):

- While holding the centroids (μ) fixed, we choose the assignments ($c^{(i)}$) that minimize the distance squared.
- Assigning a point to the *closest* centroid is mathematically guaranteed to minimize this part of the cost function.

2. Step 2 (Move Centroids):

- While holding the assignments (c) fixed, we choose the centroid locations (μ) that minimize the cost.
 - Setting the centroid to be the **mean (average)** of the points assigned to it is mathematically guaranteed to minimize the sum of squared distances for those points.
-

Convergence Guarantee

- Because both steps of the algorithm are designed to reduce (or maintain) the cost function J , the K-means algorithm is **guaranteed to converge**.
- The cost function will decrease with every iteration until it settles at a minimum.
- **Debugging Tip:** If you plot the cost function over iterations and see it *increasing*, there is a bug in your code.
- **Stopping Criteria:** You can declare convergence when the cost function stops decreasing (or decreases very slowly).

Initializing K-Means

The very first step of the K-means algorithm is choosing the initial locations for the cluster centroids ($\mu_1, \dots, \mu_K$). How you choose these initial values significantly affects the algorithm's performance.

1. Random Initialization Method

The most common and recommended way to initialize the centroids is to pick specific examples from your training set.

- **Constraint:** The number of clusters K must be less than the number of training examples m ($K < m$).
- **The Steps:**
 1. Randomly shuffle the training examples.
 2. Pick the first K examples from the shuffled list.
 3. Set your centroids ($\mu_1, \dots, \mu_K$) to be equal to the locations of these K examples.

2. The Problem: Local Optima

K-means is sensitive to initialization. Depending on which random points you pick, the algorithm might converge to different solutions.

- **Global Optimum:** The ideal solution where clusters are perfectly separated, resulting in the lowest possible cost (distortion).
- **Local Optima:** Sub-optimal solutions. This happens when K-means gets "stuck." For example, it might split one logical cluster into two while merging two distinct clusters into one.

Because K-means uses gradient-descent-like logic to minimize the cost function J , it can get stuck in a local minimum and fail to find the best clustering.

3. The Solution: Multiple Initializations

To solve the problem of local optima, we run the algorithm multiple times with different random starts and pick the best result.

The Algorithm:

1. **Loop** (e.g., 100 times):
 - **Step A:** Randomly initialize the centroids (pick K random examples).
 - **Step B:** Run the K-means algorithm (Assign & Move steps) until it converges.

- **Step C:** Compute the **Cost Function (Distortion)** J for this specific run.

2. Select the Best Run:

- Compare the cost function J from all 100 runs.
- Pick the clustering assignment that resulted in the **lowest** cost J .

Typical Usage:

- Performing **50 to 1,000** random initializations is common.
- Doing this virtually guarantees that you find a good solution (the global optimum) rather than a poor local optimum.
- *Note:* Running this more than 1,000 times usually has diminishing returns and becomes computationally expensive.

Choosing the Number of Clusters (K)

One of the most common questions in K-means is: "**How do I choose the value of K ?**"

Because clustering is unsupervised, there is often no single "right" answer. The data itself might be ambiguous—one person might see 2 clusters, while another sees 4.

However, there are two main methods to help you decide.

1. The Elbow Method (Academic Approach)

This is a technique to automatically visualize the "best" K , though it is not always reliable.

- **How it works:**

1. Run K-means with a variety of K values (e.g., $K = 1, 2, 3, \dots, 10$).
2. For each K , calculate the final **Cost Function** (Distortion) J .
3. Plot the cost J (y-axis) against the number of clusters K (x-axis).

- **What to look for:**

- As K increases, the cost J will always go down.
- You are looking for an "**Elbow**" in the curve—a point where the cost decreases rapidly and then suddenly flattens out. That point is your optimal K .

- **The Limitation:**

- In real-world applications, the curve is often very smooth with no distinct "elbow." This makes the method ambiguous and difficult to use reliably.
- **Important:** You should *never* choose K simply to minimize the cost function J . If you did, you would always choose the largest possible K (where every point is its own cluster), which is useless.

2. The "Downstream Purpose" Method (Recommended)

Since you are usually clustering data to solve a specific real-world problem, the best way to choose K is to ask: "**Which K helps me solve my problem best?**"

- **Example: T-Shirt Sizing**

- Imagine you have height and weight data for customers.
- **Option A ($K = 3$):** You create Small, Medium, and Large sizes. This is cheaper to manufacture and market.
- **Option B ($K = 5$):** You create XS, S, M, L, XL. This fits customers better but is more expensive to manufacture.
- **Decision:** You run K-means for both $K = 3$ and $K = 5$, then compare the business

trade-offs (Customer Fit vs. Manufacturing Cost) to decide.

- **Example: Image Compression**

- You can use K-means to compress an image by reducing the number of colors.
 - **Trade-off:** You balance **Image Quality** (higher K) vs. **File Size** (lower K).
-

Summary of K-Means

Congratulations! You have finished the section on K-means clustering. You now know:

- How the algorithm works (Assign points → Move centroids).
- The optimization objective (Cost function J).
- How to initialize it (Random initialization).
- How to choose K (Downstream purpose).